Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу «Операционные системы»

Группа: М8О-214БВ-25

Студент: Богатырева Е.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.11.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создаёт дочерний процесс. Родительский и дочерний процессы представлены разными программами. Взаимодействие между процессами осуществляется через разделяемую память (POSIX Shared Memory). Пользователь вводит команды вида: «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным. *Ограничение:* запрещено использовать устаревший SysV API (shmget, shmat), необходимо использовать shm_open и mmap.

Общий метод и алгоритм решения

Использованные системные вызовы и функции:

- `int shm_open(const char *name, int oflag, mode_t mode);` – создаёт (родитель, флаги O_CREAT | O_EXCL) или открывает (потомок) именованный объект разделяемой памяти POSIX.
- `int ftruncate(int fd, off_t length);` – задаёт размер объекта разделяемой памяти, равный размеру структуры shared_t.
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);` – отображает объект в адресное пространство процесса (PROT_READ | PROT_WRITE, MAP_SHARED), благодаря чему оба процесса работают с одними и теми же физическими страницами.
- `int munmap(void *addr, size_t length);` – снимает отображение памяти.
- `int shm_unlink(const char *name);` – удаляет имя объекта разделяемой памяти из системы.
- `pid_t fork(void);` – создаёт дочерний процесс.
- `int execl(const char *path, const char *arg, ...);` – заменяет образ памяти дочернего процесса программой child.
- `pid_t waitpid(pid_t pid, int *status, int options);` – ожидает завершения дочернего процесса и получает код его завершения.
- `int close(int fd);` – закрывает файловый дескриптор.
- `void exit(int status);` и `void _exit(int status);` – завершают процесс с заданным статусом (_exit используется в потомке при ошибке execl).
- `int sem_init(sem_t *sem, int pshared, unsigned int value);` – инициализирует неименованный семафор POSIX; при pshared = 1 семафор может использоваться разными процессами и размещается в разделяемой памяти.
- `int sem_wait(sem_t *sem);` – уменьшает значение семафора (операция P), при нулевом значении блокирует процесс.
- `int sem_post(sem_t *sem);` – увеличивает значение семафора (операция V) и будит ожидающий процесс.

- `int sem_destroy(sem_t *sem);` – уничтожает неименованный семафор.
- Вспомогательные функции стандартной библиотеки: `getline` (чтение строк со стандартного ввода), `fgets` (чтение имени файла), `strtod` (разбор числа с плавающей точкой), `fopen`/`fprintf`/`fflush` (запись результата в файл), `perror` (вывод описания системной ошибки).

Общая схема работы

Программа состоит из трёх файлов: общего заголовка common.h, программы родительского процесса parent.c и программы дочернего процесса child.c. Обмен данными организован по классической схеме «производитель – потребитель» через кольцевой буфер, лежащий в разделяемой памяти, с синхронизацией на трёх семафорах.

В common.h описана структура shared_t, которая целиком размещается в отображаемом объекте. Она содержит: три семафора – mutex (взаимное исключение при доступе к буферу, начальное значение 1), empty (число свободных ячеек, начальное значение SLOT_COUNT) и full (число занятых ячеек, начальное значение 0); индексы head (откуда читает потомок) и tail (куда пишет родитель); флаг finished_flag, означающий, что ввод закончен; массив из SLOT_COUNT = 32 ячеек slot_t. Каждая ячейка хранит число типа double (value) и признак is_eol – маркер конца строки.

Родительский процесс (parent)

1. Принимает через argv путь к исполняемому файлу дочерней программы. Выводит приглашение и считывает с помощью fgets первую строку ввода – имя выходного файла; завершающий символ перевода строки удаляется, пустое имя считается ошибкой. Создаёт объект разделяемой памяти с помощью shm_open (O_EXCL защищает от запуска поверх уже существующего объекта), задаёт размер через ftruncate и отображает его через mmap.

2. Инициализирует три семафора функцией sem_init с параметром pshared = 1 – без него семафоры не работали бы между разными процессами. Обнуляет head, tail и finished_flag.

3. Вызывает fork. В потомке закрывается дескриптор разделяемой памяти и вызывается execl для запуска программы child, которой вторым аргументом (argv[1]) передаётся имя выходного файла; если execl не удался, потомок завершается со статусом 127.

4. Выводит приглашение «Enter numbers» и в цикле читает остальные строки со стандартного ввода функцией getline. Каждая строка разбивается на токены функцией strtod; корректные числа помещаются в буфер функцией push с is_eol = 0. Некорректный токен пропускается с сообщением в stderr. После последнего числа строки в буфер помещается маркер конца строки (is_eol = 1).

5. Функция push сначала выполняет sem_wait(empty) – если свободных ячеек нет, родитель блокируется. Затем под семафором mutex записывает ячейку в позицию tail, сдвигает tail по модулю SLOT_COUNT, освобождает mutex и выполняет sem_post(full), сообщая потомку о новой ячейке. Порядок захвата (сначала empty, потом mutex) важен: иначе при полном буфере можно заблокироваться, удерживая mutex, и получить взаимную блокировку.

6. После окончания ввода (EOF) родитель вызывает push_final: в той же критической секции, в которой кладёт в буфер завершающую ячейку-маркер, он устанавливает finished_flag = 1. Затем ожидает потомка через waitpid, выводит его код завершения, уничтожает семафоры, снимает отображение и удаляет объект через shm_unlink.

Дочерний процесс (child)

1. Проверяет, что передано имя файла (argc ≥ 2). Открывает существующий объект разделяемой памяти через shm_open (без O_CREAT) и отображает его через mmap. Открывает на запись файл с именем argv[1].

2. В цикле выполняет sem_wait(full) – ожидает появления данных, затем под семафором mutex копирует ячейку из позиции head и сдвигает head. Если извлечённая ячейка является маркером конца строки, finished_flag установлен и буфер после извлечения пуст (head == tail), то это завершающий маркер: потомок выходит из цикла, ничего не записывая. Иначе освобождает mutex, выполняет sem_post(empty), уведомляя родителя о свободном месте, и обрабатывает ячейку уже вне критической секции.

3. Если ячейка содержит число – прибавляет его к накопленной сумме. Если ячейка является маркером конца строки – записывает сумму в выходной файл (формат %.10g) и сбрасывает накопитель. Для строки без чисел (пустой или состоящей только из некорректных токенов) в файл ничего не записывается.

4. После завершения цикла закрывает файл, снимает отображение и закрывает дескриптор. Удаление объекта разделяемой памяти выполняет только родитель.

Обработка ошибок

Все системные вызовы shm_open, ftruncate, mmap, fork, waitpid, а также fopen в потомке, проверяются на ошибку; также проверяется успешность чтения имени файла функцией fgets. При сбое выводится описание через perror и процесс завершается с кодом EXIT_FAILURE. Проверяется число аргументов командной строки родителя и потомка, а пустое имя файла отклоняется. Невозможность создать выходной файл приводит к завершению потомка с кодом 1. Ошибка запуска дочерней программы (execl) приводит к завершению потомка с кодом 127, который родитель выводит на экран.

Код программы

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <semaphore.h>

#define SHM_NAME    "/lab3_var2_shm"
#define SLOT_COUNT  32

typedef struct {
    double value;
    int    is_eol;
} slot_t;

typedef struct {
    sem_t   mutex;
    sem_t   empty;
    sem_t   full;

    int     head, tail;
    int     finished_flag;
    slot_t  slots[SLOT_COUNT];
} shared_t;

#endif
```

parent.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <semaphore.h>
#include "common.h"

static shared_t *shm = NULL;
static int shm_fd = -1;

static void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

static void cleanup(void) {
    if (shm) { munmap(shm, sizeof(shared_t)); shm = NULL; }
    if (shm_fd >= 0) {
        close(shm_fd);
        shm_unlink(SHM_NAME);
        shm_fd = -1;
    }
}

static void push(double v, int is_eol) {
    sem_wait(&shm->empty);
    sem_wait(&shm->mutex);
```

```
    shm->slots[shm->tail].value  = v;
    shm->slots[shm->tail].is_eol = is_eol;
    shm->tail = (shm->tail + 1) % SLOT_COUNT;

    sem_post(&shm->mutex);
    sem_post(&shm->full);
}

static void push_final(void) {
    sem_wait(&shm->empty);
    sem_wait(&shm->mutex);

    shm->slots[shm->tail].value  = 0.0;
    shm->slots[shm->tail].is_eol = 1;
    shm->tail = (shm->tail + 1) % SLOT_COUNT;
    shm->finished_flag = 1;

    sem_post(&shm->mutex);
    sem_post(&shm->full);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <child_binary>\n", argv[0]);
        return EXIT_FAILURE;
    }

    char fileName[1024];
    printf("Enter output file name: ");
    if (fgets(fileName, sizeof(fileName), stdin) == NULL) {
        fprintf(stderr, "Error reading file name\n");
        return EXIT_FAILURE;
    }
    size_t len = strlen(fileName);
    if (len > 0 && fileName[len - 1] == '\n') fileName[len - 1] = '\0';
    if (strlen(fileName) == 0) {
        fprintf(stderr, "Empty file name\n");
        return EXIT_FAILURE;
    }

    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR | O_EXCL, 0600);
    if (shm_fd < 0) die("shm_open");
    if (ftruncate(shm_fd, sizeof(shared_t)) < 0) die("ftruncate");

    shm = mmap(NULL, sizeof(shared_t), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shm == MAP_FAILED) die("mmap");

    if (sem_init(&shm->mutex, 1, 1) < 0) die("sem_init mutex");
    if (sem_init(&shm->empty, 1, SLOT_COUNT) < 0) die("sem_init empty");
    if (sem_init(&shm->full,  1, 0) < 0) die("sem_init full");

    shm->head = shm->tail = 0;
    shm->finished_flag = 0;

    pid_t pid = fork();
    if (pid < 0) die("fork");
    if (pid == 0) {
        close(shm_fd);
        execl(argv[1], argv[1], fileName, (char *)NULL);
        perror("execl");
        _exit(127);
    }
```

```
    char *line = NULL;
    size_t cap = 0;
    ssize_t bytes_read;

    printf("Enter numbers (Ctrl+D to exit):\n");
    while ((bytes_read = getline(&line, &cap, stdin)) > 0) {
        char *p = line;
        while (*p) {
            while (*p == ' ' || *p == '\t' || *p == '\r') p++;
            if (*p == '\0' || *p == '\n') break;

            char *end = NULL;
            errno = 0;
            double v = strtod(p, &end);
            if (end == p) {
                fprintf(stderr, "[parent] bad token, skipping\n");
                while (*p && *p != ' ' && *p != '\t' && *p != '\n') p++;
                continue;
            }
            push(v, 0);
            p = end;
        }
        push(0.0, 1);
    }
    free(line);

    push_final();

    int status = 0;
    if (waitpid(pid, &status, 0) < 0) die("waitpid");

    if (WIFEXITED(status))
        printf("[parent] child exited with code %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[parent] child killed by signal %d\n", WTERMSIG(status));

    sem_destroy(&shm->mutex);
    sem_destroy(&shm->empty);
    sem_destroy(&shm->full);

    cleanup();
    return EXIT_SUCCESS;
}
```

child.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include "common.h"

static shared_t *shm = NULL;
static int shm_fd = -1;
static FILE *out_fp = NULL;

static void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}
```

```
static void cleanup(void) {
    if (out_fp) { fclose(out_fp); out_fp = NULL; }
    if (shm) { munmap(shm, sizeof(shared_t)); shm = NULL; }
    if (shm_fd >= 0) { close(shm_fd); shm_fd = -1; }
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Error: file name argument required\n");
        return EXIT_FAILURE;
    }

    shm_fd = shm_open(SHM_NAME, O_RDWR, 0);
    if (shm_fd < 0) die("shm_open (child)");

    shm = mmap(NULL, sizeof(shared_t), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shm == MAP_FAILED) die("mmap (child)");

    out_fp = fopen(argv[1], "w");
    if (!out_fp) die("fopen result file");

    double sum = 0.0;
    int have_numbers = 0;

    for (;;) {
        sem_wait(&shm->full);
        sem_wait(&shm->mutex);

        slot_t s = shm->slots[shm->head];
        shm->head = (shm->head + 1) % SLOT_COUNT;

        if (s.is_eol && shm->finished_flag && (shm->head == shm->tail)) {
            sem_post(&shm->mutex);
            sem_post(&shm->empty);
            break;
        }

        sem_post(&shm->mutex);
        sem_post(&shm->empty);

        if (s.is_eol) {
            if (have_numbers)
                fprintf(out_fp, "%.10g\n", sum);
            fflush(out_fp);
            sum = 0.0;
            have_numbers = 0;
        } else {
            sum += s.value;
            have_numbers = 1;
        }
    }

    cleanup();
    return EXIT_SUCCESS;
}
```

Сборка

```
gcc -Wall -Wextra -o parent parent.c -pthread -lrt
gcc -Wall -Wextra -o child child.c -pthread -lrt
```

Протокол работы программы

Тест 1: обычные строки, пустая строка, некорректный токен

```
$ cat in.txt
out.txt
1.5 2.5 3
10 20 30.25

-4.5 4.5
abc 5 6
7
$ ./parent ./child < in.txt
Enter output file name: Enter numbers (Ctrl+D to exit):
[parent] bad token, skipping
[parent] child exited with code 0
$ cat out.txt
7
60.25
0
11
7
```

Суммы совпадают с ожидаемыми: 1.5 + 2.5 + 3 = 7; 10 + 20 + 30.25 = 60.25; для пустой строки в файл ничего не записано; -4.5 + 4.5 = 0; токен «abc» пропущен, 5 + 6 = 11; 7. Имя файла out.txt, введённое первой строкой, использовано потомком.

Тест 2: имя файла sums.txt и 200 строк, в строке с номером i записано i единиц

```
$ ./parent ./child < bigin.txt
Enter output file name: Enter numbers (Ctrl+D to exit):
[parent] child exited with code 0
$ wc -l sums.txt
200 sums.txt
$ head -3 sums.txt
1
2
3
$ tail -2 sums.txt
199
200
```

Суммарно через буфер из 32 ячеек пройдено 20 100 чисел, то есть буфер заполнялся многократно; синхронизация на семафорах отработала без потерь и взаимных блокировок. Тест был повторён 30 раз подряд с одинаковым результатом.

Тест 3: после имени файла нет ни одной строки

```
$ printf 'empty.txt\n' | ./parent ./child
Enter output file name: Enter numbers (Ctrl+D to exit):
[parent] child exited with code 0
$ wc -c empty.txt
0 empty.txt
```

Тест 4: ошибки

```
$ ./parent
Usage: ./parent <child_binary>
$ printf '\n' | ./parent ./child
Enter output file name: Empty file name
$ printf 'x.txt\n1 2\n' | ./parent ./nonexist
Enter output file name: Enter numbers (Ctrl+D to exit):
execl: No such file or directory
[parent] child exited with code 127
$ printf '/nonexistent_dir/x.txt\n1 2\n' | ./parent ./child
Enter output file name: Enter numbers (Ctrl+D to exit):
fopen result file: No such file or directory
```

```
[parent] child exited with code 1
```

Вывод

В ходе лабораторной работы была реализована программа из двух процессов, обменивающихся данными через отображаемую разделяемую память (shm_open + mmap) вместо каналов. Для корректной работы кольцевого буфера использованы три неименованных семафора POSIX (mutex, empty, full), инициализированные с флагом pshared = 1 и размещённые непосредственно в разделяемой памяти. Программа правильно вычисляет суммы вещественных чисел для строк произвольной длины, принимает имя выходного файла первой строкой ввода, пропускает пустые строки и некорректные токены, а также проверяет коды возврата системных вызовов. Основная сложность заключалась в организации синхронизации между разными процессами: необходимо было корректно инициализировать примитивы с флагом общего доступа, корректно сообщить потомку о завершении ввода (завершающая ячейка-маркер и флаг finished_flag устанавливаются родителем в одной критической секции, иначе потомок мог бы либо зависнуть, либо потерять последнюю строку) и освободить разделяемый объект через shm_unlink.